# Machine Learning Course Project 1

VehicleVision Classification
&
Flight Price Prediction

**Team name  IDK_ML**


**Team members**

Hitesh Dahiya (MT2026037)

Priti Aher (MT2026075)

Shubhranshu Gupta (MT2026096)

Anjali Kiran (MT2026124)


15 October 2026

## Project overview

This course project applies supervised machine learning to two prediction problems: identifying vehicle types and estimating flight ticket prices. In both projects, the training data contains known outcomes, while the test data requires predictions. The work covers data inspection, preprocessing, feature engineering, model training, validation, and preparation of Kaggle submission files.

VehicleVision is a classification problem. Each vehicle must be assigned to one of three classes: Sedan, SUV, or Sports Car. The inputs are numerical measurements such as power, weight, dimensions, fuel efficiency, and performance. The experiments compare Logistic Regression, bagging, and a larger hybrid that combines tree ensembles with LR. Duplicate measurements are an important data issue, so the report distinguishes the original validation split from the grouped split used after cleaning.

Flight Price Prediction is a regression problem because the output is a numerical ticket price in rupees. The data includes airline, route, cabin class, travel duration, time categories, and days remaining before departure. Missing airline and duration values require preprocessing. The experiments use a Ridge-based regression pipeline, Random Forest, Extra Trees, and several ways of combining their predictions.

The model comparisons use accuracy for vehicle classification and RMSE, MAE, and $R^2$ for flight prices. Validation procedures are explained alongside the results because scores obtained from different splits cannot always be compared directly. The report also describes the main parameter settings, submission checks, and limitations of the experiments.

For VehicleVision, LR alone and LR plus bagging both achieved 90.40% public leaderboard accuracy. LR alone is the recommended final choice because it also performed slightly better on the common local validation split. For flight prices, the submitted geometric ensemble achieved a public $R^2$ of 0.96181. The arithmetic blend was slightly stronger locally, so the local comparison and the submitted model are described separately. Public leaderboard results are kept distinct from local validation and do not establish private-test performance.

The following two parts present each project separately, beginning with the problem and data, followed by preprocessing, models, results, and the final submission. The shared GitHub repository is listed at the end of the report.

# Part 1 VehicleVision Classification

Classification of Sedan SUV and Sports Car

## 1 Problem and data

The task was to classify each vehicle as Sedan (0), SUV (1), or Sports Car (2). The training file contained 10,000 rows and 22 columns, including vehicle_id and the target vehicle_class. The test file contained 2,500 rows and 21 columns. We kept vehicle_id for the submission and excluded it from training, leaving 20 numerical inputs.

## 2 Data inspection

The class counts were fairly balanced: 3,389 Sedans, 3,326 SUVs, and 3,285 Sports Cars. The initial checks included the data shape, sample rows, target values, and class counts. In the simple ensemble, removing repeated feature-and-label rows reduced the training set from 10,000 to 7,500 rows.

## 3 Preprocessing

The standalone Logistic Regression (LR) model used median imputation followed by StandardScaler. Imputation filled missing numerical values, while scaling put the inputs on comparable scales. These steps were fitted together with LR in a pipeline.

The simple ensemble replaced infinite values with missing values, removed duplicates, and checked the input columns and target labels. LR used imputation and scaling; bagging used imputation without scaling. The engineered hybrid retained all 10,000 rows and added 17 features. Its LR branch used scaling, but the notebook did not include a general imputation step.

# VehicleVision models

## 4 Feature engineering

The hybrid used 37 predictors: the original 20 features and 17 derived features. These were intended to express relationships between specifications rather than treat every measurement separately.

The powertrain features were power_to_weight, torque_to_weight, battery_to_weight, battery_to_accel, power_per_cyl, and torque_per_disp. Geometry features were volume_est, density_est, wheelbase_to_length, and length_to_width. The aerodynamic features were frontal_area and drag_area. The remaining features were kinetic_energy, braking_force, avg_fuel_efficiency, fuel_tank_to_weight, and low_slung_ratio.

For example, power_to_weight divided horsepower by vehicle weight. Volume was estimated from length, width, and height with a unit conversion. Kinetic energy assumed a speed of 100 km/h, and braking_force divided this estimate by braking distance. These were numerical proxies, not new measurements. Some ratios added 1e-5 to the denominator.

## 5 Models tried

Standalone LR used C = 1.0, max_iter = 3000, and the lbfgs solver. The simple ensemble used the same LR settings and combined its probabilities with bagging through soft voting. LR received 95% weight and bagging 5%. The bagging model used 80 trees, no depth limit, minimum leaf size 5, sample fraction 0.8, all features, bootstrap sampling, and seed 42.

The engineered hybrid combined four tree ensembles with LR. Gini bagging and entropy bagging each used 300 trees, maximum depth 16, minimum leaf size 2, and feature fraction 0.85. Their seeds were 42 and 101. Random Forest and Extra Trees each used 500 trees, minimum split size 4, and seed 42. The LR branch used C = 1.0, max_iter = 2000, and seed 42.

The four tree probability outputs were averaged equally. This average received 40% weight, while LR received 60%. Each tree branch therefore contributed 10% to the final probabilities. The predicted class was the class with the highest combined probability. The weights were fixed in the code; no weight search was run in these notebooks.

## 6 Validation method

Standalone LR used a stratified 80/20 split with seed 42, giving 8,000 training rows and 2,000 validation rows. The simple ensemble grouped identical measurement vectors and used the first split from a five-split StratifiedGroupKFold with seed 2026. This was one grouped validation check, not the average of five runs. LR, bagging, and their blend were evaluated on the same 1,500 rows.

# VehicleVision results

## 7 Results and comparison

| Model | Validation accuracy | Evaluation |
|---|---|---|
| Standalone LR | 88.35% | Raw stratified split |
| LR in simple ensemble | 87.73% | Cleaned grouped split |
| Bagging | 87.13% | Cleaned grouped split |
| 95% LR and 5% bagging | 87.67% | Cleaned grouped split |

LR performed slightly better than bagging and their blend on the common grouped split. The standalone score of 88.35% came from different data and a different split, so it cannot show whether duplicate removal helped or hurt. The larger hybrid had no saved validation score and could not be ranked in this comparison.

| Class | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| Sedan | 0.83 | 0.83 | 0.83 | 521 |
| SUV | 0.88 | 0.91 | 0.89 | 495 |
| Sports Car | 0.93 | 0.90 | 0.92 | 484 |

The simple ensemble had its highest displayed recall for SUVs and highest precision for Sports Cars. The hybrid predicted 33.36% SUVs, 33.32% Sedans, and 33.32% Sports Cars on the test set. These percentages describe the prediction mix, not accuracy.

## 8 Public score and final choice

LR alone and the combined LR plus bagging model both achieved a public Kaggle accuracy of 0.90400 (90.40%). LR alone is the recommended final choice because it matched the public score and was slightly better in the local comparison. A public tie does not guarantee the same result on the private test data. No public score was recorded for the engineered hybrid.

## 9 Submission and conclusion

Standalone LR was refitted on all 10,000 rows and saved vehicle_submission.csv. The hybrid used all 10,000 rows with 37 features and saved submission_hybrid_super_blend.csv. The simple ensemble used the 7,500 cleaned rows and saved submission_simple_lr_bagging.csv. Each file contained vehicle_id and vehicle_class for all 2,500 test rows in their original order.

The simple ensemble checked column consistency, target values, group separation, submission size, unique IDs, missing values, predicted classes, and ID order. The other notebooks displayed previews or class counts. Overall, the recorded comparison supports LR as the final choice; the larger hybrid needs validation before claiming an improvement.

# Part 2 Flight Price Prediction

Regression for flight ticket prices

## 1 Problem and data

The aim was to predict ticket prices in rupees. The training data had 15,589 rows and 12 columns; the test data had 3,898 rows and 11 columns. Inputs described the airline, flight, cities, departure and arrival time categories, stops, cabin class, duration, and days left before departure. The ID was kept for submission and excluded from the predictors.

## 2 Exploratory analysis

Airline was missing in 3,089 training rows and 799 test rows. Duration was missing in 3,095 training rows and 776 test rows. Dropping these rows would remove a substantial part of the data, so missing values were filled instead. The data contained 1,070 flight codes, 18 airlines, six source cities, six destination cities, and two cabin classes.

Training durations ranged from 0.92 to 38.08 hours, compared with 0.92 to 29.08 hours in the test set. Days left ranged from 1 to 57 in both sets. The notebook also plotted price distribution, price by cabin class, and price against days left. These checks motivated log-price training for the forests, separate LR models by cabin and carrier group, and booking-time features. They do not prove that earlier booking always causes a lower price.

## 3 Preparing inputs for the tree models

Missing airline values were recovered using flight-prefix mappings. Duration was filled in stages using flight and route medians, flight medians, route and stops medians, and finally the global median. The models used 32 inputs: 11 categorical and 21 numerical features.

The features included airline and route combinations, cabin and stops, ordered time categories, duration, and a direct-flight flag. Booking-time inputs included days left, its log and inverse, and indicators for 2 days or fewer, 5 days or fewer, and 25 days or more. Relative-duration features included route minimum duration, excess duration, duration per stop, and a duration ratio. Frequency counts described flight, airline, route, and airline-route occurrence in the training data.

The categorical features were target-encoded with smoothing. Training encodings were cross-fitted, and validation and test encodings used the relevant training labels. Cabin, stops, and time categories also had numerical representations. The forests did not use feature scaling and were trained on log(1 + price). Flight identity supported duration recovery and counts but was not directly target-encoded.

Some airline and duration mappings used combined training and application features, while frequency counts used all supplied training rows. Application prices were not used, but preprocessing was not fully isolated within each validation fold. Frequency and duration features are dataset summaries and proxies, rather than direct measures of demand or connection time.

# Flight regression pipeline

## 4 Preparing inputs for LR

The LR pipeline used flight prefix as the carrier input. Missing duration was filled using source, destination, and stops medians, followed by a global median. A separate flag marked missing duration. Carrier, cities, time categories, and stops were one-hot encoded. The five numerical inputs were duration, days left, log days left, a last-three-days flag, and the missing-duration flag.

Separate models were fitted for Economy, Business tickets with prefixes 6E, AI, and IX, and other Business carriers. These were carrier-based groups. The model also used comparable fares from matching training itineraries: median, lower quartile, minimum, and mean prices. If close matches were unavailable, it fell back through flight-route, flight, carrier-route, and segment statistics.

Numerical inputs were scaled, pairwise interactions were added, and the resulting features were scaled again before Ridge regression. Ridge applies a penalty to keep coefficients from becoming too large. Smaller Ridge models then corrected the remaining errors using flight and increasingly specific route, cabin, and days-left keys.

Four variants with different fare statistics, penalties, and correction stages were averaged. Two nearby-day estimates also used comparable training fares, weighted by exp(-absolute days-left difference / 5). Similar booking horizons therefore received more weight. Nonnegative least squares combined these estimates with the four-variant prediction, separately by segment and availability of comparable itineraries.

Grouped fare statistics excluded the current training row when matching peers existed. The final segment fallback included all segment prices. Nearby days referred to days remaining before departure, not calendar dates.

## 5 Models and settings

Random Forest and Extra Trees each used 180 trees, maximum depth 25, minimum split size 6, minimum leaf size 2, and feature fraction 0.75. Five shuffled folds used seed 42; model seeds were 42 plus the fold index. These were fixed settings rather than a documented hyperparameter search. Extra Trees used no bootstrap sampling by default.

The RF plus ET pipeline averaged the two forest predictions in log-price space and converted them back to rupees. Despite the DT name in the project, there was no separately fitted single decision tree. Test predictions were averaged across the five fold models in price space.

## 6 Prediction checks

Each LR variant was clipped between its segment training minimum and 1.1 times the segment maximum before averaging. Forest test predictions were clipped to the training price range. Submission checks covered row count, finite positive prices, and unique IDs.

# Flight evaluation

## 7 Validation method

The forest results used out-of-fold predictions: each row was predicted by a model that had not trained on its price. The LR notebook defined five-fold development evaluation and an 80/20 holdout, but did not contain saved numerical results for those checks.

The ensemble comparison used 9,353 rows for training, 3,118 for calibration, and 3,118 for evaluation. Calibration data was used to learn combination weights. Evaluation labels were removed before prediction and used only for scoring. Internal LR stacking weights were learned from out-of-fold predictions within the supplied training portion.

The LR development stack was not fully nested. Models creating stacking inputs for other folds may have seen the held-out fold, so that development calculation should not be treated as an independent final test.

## 8 Model scores

RMSE gives more weight to large errors, while MAE is the average absolute error. Both are in rupees and lower values are better. Higher $R^2$ is better. Results from different validation procedures should be compared with care.

| Model | RMSE | MAE | $R^2$ |
|---|---|---|---|
| Main LR | 9,682.5590 | 3,447.2779 | 0.9335 |
| Random Forest | 10,425.3633 | 3,207.6640 | 0.9290 |
| Extra Trees | 10,329.7524 | 3,141.5442 | 0.9303 |
| RF plus ET | 10,312.5195 | 3,154.7929 | 0.9305 |
| Geometric ensemble | 9,537.6565 | 3,203.7948 | 0.9354 |

The forest and RF plus ET scores above came from five-fold out-of-fold evaluation. Main LR and the geometric ensemble were scored on the ensemble evaluation split. The same-split comparison on the next page gives a more direct view of the effect of combining the models.

## 9 Ways of combining predictions

We compared equal arithmetic and geometric averages, a fitted global weight, separate segment weights, nonnegative least squares (NNLS), and Ridge stacking. The global fitted LR weight was 0.3128, leaving about 0.6872 for RF plus ET. Segment weights fell back to the global weight when fewer than 100 calibration rows were available.

NNLS allowed nonnegative coefficients without requiring them to sum to one. Ridge stacking used an intercept and alpha = 10, with its two prediction inputs scaled to thousands of rupees. These learned combinations were compared with the fixed blends; they were not the method used for the final export.

## Flight results and final submission

### 10 Comparison on the same evaluation rows

All results in this table used the same 3,118 evaluation rows, so they can be compared directly.

| Method | RMSE | MAE | $R^2$ |
|---|---|---|---|
| Arithmetic 50/50 | 9,532.9230 | 3,207.8193 | 0.9355 |
| Geometric 50/50 | 9,537.6565 | 3,203.7948 | 0.9354 |
| Weighted average | 9,608.1868 | 3,159.4860 | 0.9345 |
| NNLS stacking | 9,639.0888 | 3,116.0775 | 0.9341 |
| Segment weighted | 9,640.5957 | 3,169.3990 | 0.9340 |
| Ridge stacking | 9,644.4166 | 3,182.1596 | 0.9340 |
| LR only | 9,682.5590 | 3,447.2779 | 0.9335 |
| RF plus ET only | 9,888.5784 | 3,175.7278 | 0.9306 |

Both equal-weight blends improved RMSE and $R^2$ over the individual pipelines. Arithmetic averaging had the lowest local RMSE, about Rs 4.73 lower than the geometric blend. NNLS gave the lowest MAE but a higher RMSE. More complicated weighting did not improve RMSE in this comparison.

### 11 Final model and public score

The exported model used the geometric 50/50 blend. It took the square root of the product of the LR prediction and the RF plus ET prediction. The component pipelines were fitted using the full training data, and 3,898 predicted prices were written to data/submission_ensemble_simple.csv.

The ensemble submission achieved a public leaderboard $R^2$ of 0.96181. Its local $R^2$ was 0.9354; these scores came from different data. The local results support combining the models, although they do not show that geometric averaging was better than arithmetic averaging.

### 12 Conclusion

For flight prices, combining LR with the tree pipeline reduced local prediction error compared with either pipeline alone. The final export used the geometric blend. For VehicleVision, the small bagging contribution did not improve the local LR result, and both submissions tied publicly. The two projects therefore led to different model choices based on their recorded results.

### Project repository

Both projects, including their notebooks and implementation, are available in the repository below.

https://github.com/shubranshugupta/ml-project1